



MODULE 9

Testing & Debugging for iOS

Enterprise Mobile Application Development Bootcamp

Phase 02 | iOS Development with Swift

Module 9 Roadmap

Six blocks. By the end: a fully tested, debugged PNC Mobile feature, ready for the capstone.



The Testing Pyramid & XCTest

- Why Test at All
- The Testing Pyramid
- XCTest & the AAA Pattern
- Live TDD Mini-Demo



The iOS Testing Pyramid

UI Tests

3–10 tests covering critical paths

Integration Tests

Repository + real Core Data, mocked network

Unit Tests

ViewModel · Service · Model - fast, isolated, many

< 1ms

Typical unit test run time

30+ sec

Typical UI test run time

70%+

Target ViewModel/Service coverage



Using Swift Testing

- Swift Testing is the modern approach to unit testing in Swift applications
- Tests can be written in classes, structs, and actors
- Tests are functions that are marked with the @Test wrapper
- The framework has built-in concurrency support

Code Bite

```
import Testing
@testable import YourAppTarget

struct CalculatorTests {

    @Test func testAddition() {
        let calculator = Calculator()
        let result = calculator.add(5, 3)
        #expect(result == 8)
    }
}
```



@testable

- Importing a module typically gives your code access to the public assets from that module
- Module assets declared as internal will be visible only within that module
 - They will not be visible to the code that is importing the module
- Occasionally you may need to access those internal assets during unit tests
- Adding the @testable wrapper to the import will grant access to those internal assets
 - Provided the module was compiled with the enable-testing flag
 - Xcode will handle this automatically for debug and test builds
- This should be used sparingly
 - Tests should primarily focus on the public interface of components



The AAA Pattern

- Tests frequently need three phases
 - Setup initial conditions
 - Execute the code under test
 - Verify the results
- Those steps are often called:
 - Arrange
 - Act
 - Assert
- Returning a Boolean success value from a test would be too limiting
 - Why did the test fail?
 - How did it fail?

Code Bite

```
// Arrange
let mock = MyMockService()
let comp = SomeComponent(svc: mock)

// Act
let result = comp.performAction()

// Assert
#expect(result == 4)
```



Assertions

- Assertions are statements about what should be the case after the code under test executes
- `#expect` is a built-in macro that checks whether a condition or expression is true
- It features soft assertions
 - A failure is recorded but it doesn't stop the test execution
- It features expression capture
 - Expression values are stored and output so you can better see why it failed
- `#expect(throws:)` can verify that a error is thrown

Code Bite

```
#expect(result == true)

#expect(total > 0)

#expect(!name.isEmpty,
        "name should always be supplied")

#expect(throws: ValidationError.tooSmall)
```



XCTest: the Legacy Approach

- XCTest is the older testing module for Swift
- Classes that contain unit tests must inherit from XCTestCase
- Every method in those classes whose name starts with lowercase "test" will be executed as unit tests
- The XCTAssertEqual function is executed to make assertions
- This framework does NOT support concurrency out of the box
 - It must be configured, if desired

Code Bite

```
class TransferServiceTests: XCTestCase {  
    func test_transferBelowBalance_succeeds() {  
        let fakeRepo = FakeAccountsRepository()  
        let viewModel = TransferViewModel(  
                                repository: fakeRepo)  
        let account = Account(id: "1",  
                                name: "Checking",  
                                maskedNumber: "0000", balance: 500)  
        await viewModel.attemptTransfer(amount: 100,  
                                        from: account, to: account)  
        XCTAssertEqual(fakeRepo.transferCallCount, 1)  
    }  
}
```



Live TDD Mini-Demo: Red, Then Green

- 1 Write the test FIRST, for a function that doesn't exist yet - it won't even compile
- 2 Write just enough of a stub to make it compile, then run it - RED (test fails)
- 3 Write the minimum implementation needed to make the assertion true
- 4 Run again - GREEN (test passes)
- 5 Refactor with confidence - the test now guards against regressions



Block 1 - The Testing Pyramid and Swift Testing

Skill Check ✓

- Why is the testing pyramid bottom-heavy - why not write mostly UI tests since they test the most realistic scenario?
- Name the three sections of the AAA pattern in order.
- In TDD, what does 'red' mean, and what should you do immediately after seeing it?



Async Testing & Test Doubles

- Testing async/await Code
- Fakes, Stubs & Mocks
- When Each Type Is Appropriate



Testing async/await Code

```
@Test func loadAccountsPopulatesViewModelFromRepository() async {  
    let fake = FakeAccountsRepository()  
    fake.stubbedAccounts = [Account(id: "1", name: "Checking", maskedNumber: "0000",  
        balance: 500)]  
    let viewModel = AccountsViewModel(repository: fake)  
  
    await viewModel.loadAccounts()    // just await it, like any other call  
  
    #expect(viewModel.accounts.count, 1)  
}  
  
// No special async test framework needed - Swift Testing supports async test  
// methods natively.
```



Fakes, Stubs & Mocks: the Vocabulary



Fake

A working, simplified implementation - e.g. FakeAccountsRepository just returns an in-memory array instead of hitting a real API



Stub

Returns canned, hard-coded answers to specific calls - no real logic behind it at all



Mock/Spy

Like a stub, but also records HOW it was called, so a test can assert on interactions (e.g. 'was transfer() called exactly once?')



A Mock Class

```
final class MockAccountsRepository: AccountsRepository {  
    var stubbedAccounts: [Account] = []  
    private(set) var fetchAccountsCallCount = 0    // <- the "mock" part  
  
    func fetchAccounts() async throws -> [Account] {  
        fetchAccountsCallCount += 1  
        return stubbedAccounts  
    }  
}
```



Using a Mock in Tests

```
@Test func loadAccountsCallsRepositoryExactlyOnce() async {  
    let mock = MockAccountsRepository()  
    let viewModel = AccountsViewModel(repository: mock)  
  
    await viewModel.loadAccounts()  
  
    #expect(mock.fetchAccountsCallCount, 1)    // asserting on the INTERACTION  
}
```



Block 2 - Async Testing & Test Doubles

Skill Check ✓

- What's the one syntactic difference between testing async code and testing synchronous code in XCTest?
- What's the difference between a stub and a mock?
- Which category does the FakeAccountsRepository from Module 7/8 belong to - fake, stub, or mock?

